



Lexical Analysis



Learning Objectives

By the end of this lecture, we should understand:

- What lexical analysis is and why it is needed.
- Key terms: token, lexeme, pattern.
- Role of finite automata (DFA/NFA) in lexical analysis.
- How a lexical analyzer works internally.



What is Lexical Analysis?

Lexical analysis is the first phase of the compiler.

Its job is to take the raw source code:

```
int x = y + 10;
```

Break it into meaningful atomic units called tokens:

Break it into meaningful atomic units (token):

Lexeme	Token Type
int	KEYWORD
x	IDENTIFIER
=	OPERATOR
y	IDENTIFIER
+	OPERATOR
10	NUMBER
;	DELIMITER

Lexical Analyzer is also called “Scanner”.



Why is Lexical Analysis Needed?

- Removes whitespace and comments.
- Converts program text into a stream of tokens.
- Makes parsing easier and faster.
- Handles error detection for invalid characters.

```
int    x    = 20  ;
```

Multiple spaces → Scanner ignores them.



Basic Concepts

✓ Lexeme

Actual substring in the source code.

Example: "while", "x", "123"

✓ Token

A category or class for lexemes.

Example: IDENTIFIER, NUMBER, KEYWORD_WHILE

Tokens have form:

$\langle \text{token-name}, \text{attribute-value} \rangle$ ($\langle \text{IDENTIFIER}, x \rangle$)



Basic Concepts

✓ Pattern

A rule describing all lexemes of a token.

Example:

IDENTIFIER $\rightarrow$ letter (letter | digit)*

NUMBER $\rightarrow$ digit+



What Does the Lexical Analyzer Do Internally?

- ◆ Input: Raw program characters
- ◆ Output: Token stream for parser



What Does the Lexical Analyzer Do Internally?

Internally, the scanner:

- Reads characters one by one.
- Groups sequences of characters into lexemes.
- Matches them using patterns (regular expressions).
- Produces tokens.
- Ignores whitespace and comments.
- Reports lexical errors.



Finite Automata in Lexical Analysis

Lexical patterns (like identifiers, numbers) are expressed using regular expressions:

Identifier: $[a-zA-Z][a-zA-Z0-9]^*$

Number: $[0-9]^+$

These regular expressions are converted into NFA $\rightarrow$ DFA $\rightarrow$ Minimized DFA.

The DFA is what actually runs during scanning.



Example DFA for Identifiers

Identifier = letter (letter | digit)*

A simplified DFA:

Start $\rightarrow$ [letter] $\rightarrow$ ID_state $\rightarrow$ [letter/digit] $\rightarrow$ ID_state (loop)

All other inputs $\rightarrow$ reject

The scanner walks the DFA to detect when a token ends.



A Simple Example of Lexical Analysis

while count < 10

Process:

Read **w h i l e** → matches KEYWORD_WHILE

Read **whitespace** → ignore

Read **c o u n t** → matches IDENTIFIER

Read **<** → matches OP_LT

Read **10** → matches NUMBER



A Simple Example of Lexical Analysis

Token stream:

<WHILE, -> <IDENTIFIER, count> <LT, -> <NUMBER,
10>



Error Handling in Lexical Analysis

If the scanner sees something it cannot match with any pattern:

Example:

```
x = 12 @ 3;
```

@ is invalid.

Scanner should report:

Lexical Error: Invalid symbol '@' at line 1



How Lexical Analyzer and Parser Interact

Source Code → Lexical Analyzer → Tokens → Parser

- The parser consumes tokens, not characters.
- Each getNextToken() call returns the next token.

Tools: Lex / Flex

Will be checked in LAB



Summary

Lexical analysis = first phase of compiler.

Breaks input into tokens.

Uses regular expressions + finite automata.

Removes whitespace/comments.

Reports illegal characters.

Output = stream of tokens for parser.